

OPEN SOURCE. KEPT ALIVE.

Modelmirror

A zero-profit, censorship-resistant archive for open-source AI models. Permanently downloadable by anyone. Provably real, byte for byte. Built to survive a hostile state.

THREE LAYERS, FAILING APART

catalog	website + search + signed hash lists	(KB)	replaceable
files	weights by their own hash, many nets	(GB)	un-seizable
discovery	signed list of live entrances	(KB)	multi-channel

integrity = one Ed25519-signed manifest, checked offline.

1. Why this has to exist

An open-weight model is a file. You download it, run it on your own machine, and keep it after the lab that trained it changes its mind. That one property is the whole point of open weights: no login, no server-side permission, no remote kill switch. The model is yours.

A file on someone else's server has none of that. It can be deleted in an afternoon, gated behind a new account, or relicensed from open to closed. Or a government can call it dangerous, and the company holding it takes it down on its own, before a single court is involved.

It already happens. Meta's first LLaMA was pulled from GitHub and Hugging Face, and the weights survived only because they were already on torrents. Microsoft published WizardLM-2 and deleted it within hours. The Books3 dataset came down after a single legal complaint. Hugging Face keeps its own takedown log, and it already runs to dozens of entries.

Every one of those is the same failure. The weights lived in one place, so there was one place to apply pressure. Modelmirror removes that place. The goal is narrow and absolute: keep every open-source and open-weight model downloadable by anyone, anywhere, forever, with no account, and let the downloader prove every byte is identical to what was published.

This document is the protocol and the defense, written out in full.

2. What we build

Modelmirror is two things at once.

A directory of everywhere a model already lives. From day one, and without hosting a single byte, we are a searchable index of where every open-source and open-weight model can be downloaded right now. We mirror the catalogs of the major platforms (Hugging Face, ModelScope, Ollama) and the direct releases from the labs, and we index the torrents and content-addressed copies that already carry these weights. Every entry records where it came from (platform, repository, commit) and, wherever the source exposes them, the source's own file hashes. We point you straight at the original download and tell you exactly which bytes to expect. If one source dies, the directory already knows the others.

A signed archive that keeps the bytes alive. For the models most likely to vanish, we go past pointing. We pull the weights, hash and sign them, and store them across our own chunk transfer, BitTorrent, content-addressed storage, and cold storage. From then on the model stays downloadable and provable even after every original copy is gone.

The directory makes the project useful at once and keeps it honest. It indexes where files already exist. It re-hosts nothing. The archive is the promise we keep, and it grows from the most-at-risk models outward.

3. Who we build against

We design for a well-funded state that wants the project gone and will spend lawyers, money, and engineers on it. We assume it wins single battles. The design makes sure no single win is fatal.

What we assume the attacker can do, each with real precedent:

Capability	What it means	Precedent
Domain seizure	Force a registry to take or redirect a name	Operation In Our Sites, 2,713 domains; Operation Offsides, ~400 domains, 2026; physical raid on the .cat registry office, 2017
Backend discovery and seizure	Find the hidden backend behind the edges, then take the box	TLS-certificate-log mining, favicon-hash pivots, error-page address leaks
DNS or routing attack	Break a key rollover, poison a resolver, hijack a route	DENIC .de signing-key outage, 2026-05-05, took the whole TLD dark for hours
Flooding	Drown any public address; modern floods top 17 Tbps	OVH-class events
Pressure on middlemen	Lean on a host or registrar in its own country	1984 Hosting terminated XMRBazaar, 2026-06-06; ISNIC suspended a .is domain in 2014 with no court order
Declaring it dangerous	Label open weights a public-safety or national-security risk, then let the platforms enforce it for free	Sam Altman asked the US Senate to license model releases, 2023; California's SB 1047; the June 2026 federal order vetting frontier models for up to 30 days before release, which held one lab to a United States only launch and pushed another to pull models from global access
Poisoned mirrors	Run a mirror that serves tampered bytes	beaten by content addressing plus hash checks

The most dangerous attack does not look like an attack. A state never has to ban open weights. It declares them a danger to the public, writes rules whose compliance cost only a few large labs can pay, and lets the platforms do the rest. The large labs welcome it, because bureaucracy is a moat and the moat is the point. Sam Altman asked the US Senate to license model releases in 2023. California's SB 1047 would have made a company answer for any "critical harm" a derivative of its open model caused, and the cleanest way to comply is to never release the weights at all. In June 2026 the US government held a leading lab's new model to a United States only launch and ordered another to pull its models from worldwide access. None of that arrives as a takedown notice. It is a Hugging Face that quietly geofences, a registry that delists, a small lab that reads the rules and decides publishing is not worth the legal exposure. A rule that makes a safety case cost a year of lawyers does not stop OpenAI. It

stops the two-person team that trained something good on rented GPUs. And once weights are public, no developer can pull them back out of the world, which is the exact reason a copy that no policy can reach has to exist before the policy does.

What we do not promise, because saying so keeps the design honest:

- We do not survive the whole internet going dark. Nothing does.
- We do not promise perfect anonymity for downloaders. Downloads need no account and we keep almost no logs, and we publish Tor and content-addressed paths, but strong anonymity means bringing your own Tor.
- We do not vouch for a model's quality or its license. We prove only that the bytes match the signed hashes.
- We do not assume any one domain, edge, host, or person survives. Survival is a property of the whole system, never of a part.

Priority order, used to settle any conflict: keep the weights available first, then provably correct, then keep operators anonymous, then the catalog, then the domains. A poisoned model is worse than a missing one, so integrity outranks everything except raw availability.

4. The shape: three layers

The system is three layers that fail apart from each other. The attacker can only ever aim at the first, and even that is replaceable.

- **The catalog.** The website, the search, and the signed hash lists. The only part with a server to point at. It sits behind cheap edge nodes that swap out, holds no model files, and rebuilds from backup. A copy of the catalog also lives in content-addressed storage, so losing the server does not lose the map. Size: kilobytes.
- **The files.** The model weights, stored by their own hash across several networks: our own chunk transfer over HTTPS (Section 8), BitTorrent v2, and content-addressed storage (IPFS). A content address is the hash of the bytes, so the same bytes always get the same address and any change makes a new one. You cannot seize a hash. The defense here is redundancy: enough copies that no takedown reaches all of them. Size: one to a few hundred gigabytes per model.
- **Discovery.** A small signed list of the entrances that are alive right now. The client walks it until one answers. It is spread over many channels, so finding a way in is itself hard to block. Size: a few kilobytes.

Lose the catalog completely and every model is still downloadable through the file layer and still findable through discovery. The heavy bytes never touch the catalog server; they move between the file networks and the downloader directly, while the catalog moves only a few kilobytes of text.

5. The signed hash manifest

Integrity lives in one object: the **manifest**, one per model version. It is plain JSON, and it is the only thing in the system trusted to say what "correct" means.

```
{
  "model_id": "meta-llama-3.1-70b-instruct",
  "version_id": "meta-llama-3.1-70b-instruct@2024-07-23",
  "source_commit": "<git sha>",
  "total_bytes": 141505024000,
  "shard_count": 35,
  "format": "safetensors",
  "whole_model_blake3": "<hash>",
  "shards": [
    { "filename": "model-00001-of-00035.safetensors",
      "size_bytes": 4718592000, "byte_offset": 0,
      "sha256": "<hash>", "blake3": "<hash>" },
    ],
  "signature": { "algorithm": "Ed25519", "public_key": "<base64>", "value": "<base64>" }
}
```

A model has a stable id. Each snapshot is a version, tied to the exact upstream commit, so the bytes are reproducible. Every shard carries two hashes (SHA-256 and BLAKE3), and the whole model carries one (BLAKE3). SHA-256 is the universal cross-check. BLAKE3 runs above 7 GB/s on a single core, against roughly 1 to 3 GB/s for SHA-256, so checking a 140 GB model runs as fast as your disk can read it. Shards target 4 GB so they store and stream cleanly, and each one is fetched and checked on its own.

The manifest is signed once with a single Ed25519 key. The signature covers a canonical form of the JSON (RFC 8785, the JSON Canonicalization Scheme) with the signature field removed, so the website, the client, and every mirror hash the exact same bytes and agree. That canonical rule is frozen before the first signature is ever made, because changing it later would break every signature that came before.

Every client carries the public key built in and checks the signature offline, before trusting a byte. There is no skip-the-check option, because that code does not exist. A manifest that fails is rejected. A shard whose hash is wrong is thrown away and fetched again.

Content addressing makes this cheaper still. When a file is offered under a content address (an IPFS CID, or a BitTorrent v2 infohash, which is the SHA-256 over the file's piece tree), the client first compares that address to the one named in the signed manifest. If they differ, the address describes different bytes, and nothing has to be downloaded to know it is wrong. The full re-hash still runs after the fetch. Content addressing adds a guarantee. It never replaces one.

6. The key, and what happens if it is lost

One key signs every manifest. Everything above stands on keeping it secret and keeping it usable.

The private key never touches a networked machine. It is generated on an offline box and split with Shamir's secret sharing into shares held by separate people in separate countries, with backups stamped into metal. No one person can sign in the project's name, and no single subpoena can compel a signature, because no single person holds the key.

The honest limit: signing a new model still needs that key brought together. If it is lost for good, the archive freezes as a snapshot. Nothing already signed disappears. Every model already in the archive stays downloadable and stays provable under the key that signed it. A new operator would have to stand up a new key in the open and start signing forward from there. The signing key for the DNS chain (Section 9) is a different key from the one that signs models, so a compromise of one is not a compromise of the other.

7. How a model reaches you

A model is never tied to one download. The directory tracks every place a version can be fetched, and the client races them. The heavy bytes never pass through our backend. They come straight from whichever source answers, while the backend only ever moves a few kilobytes of catalog text. Each source is one row in the version's download set:

Where the bytes come from	What it is	Why it is there
Provider-direct	the original Hugging Face, ModelScope, Ollama, or lab link	fastest while it lives; always recorded so you can pull from the source
Bitmodel chunks	our own content-addressed chunks over plain HTTPS (Section 8)	any web server can serve them; outlives the provider
BitTorrent v2	existing swarms, by infohash	works with the wider swarm
Content-addressed (IPFS)	a CID, by gateway or native node	the address is the hash; no registry to seize
Mirror over HTTPS	object-storage edges	hot and fast, supports partial fetches
Cold storage (Filecoin)	long-term paid copies	survives when nothing else is funded
Tor or a mirror node	a node reached over Tor or clearnet	reachable under blocking

Every one of these is checked against the same signed manifest, so the source counts for nothing on its own. The client (Section 12) races them and keeps the first that delivers bytes that pass.

For the models we host ourselves, we keep at least three copies that do not depend on each other: a hot copy in object storage with zero egress cost, warm copies on our own chunk transfer and BitTorrent v2 across at least three separated machines, and a cold copy in long-term storage. The manifest itself is mirrored into content-addressed storage and published as a Nostr event, so the map outlives the catalog server.

8. Bitmodel: moving the weights ourselves

BitTorrent and content-addressed storage both work, but a volunteer should not need a torrent client or a special node just to help keep a model alive. So Modelmirror moves its own weights a simpler way, a protocol we call **Bitmodel**: content-addressed chunks over plain HTTPS. No tracker, no swarm protocol, no torrent. A Bitmodel source is an ordinary web server handing out files named by their hash. This is the part that lets a model live on its own, free of any provider and free of us.

It is deliberately plain:

- **A chunk is a file named by its hash.** Each version is cut into content-defined chunks (the FastCDC algorithm, 4 MB at the smallest, 16 MB on average, 64 MB at the largest). The cut points follow the bytes themselves, so the same run of weights always cuts the same way. The chunk's name is its SHA-256 hash, so a chunk proves what it is. A 1 TB model lands at roughly 15,000 to 250,000 chunks, each one independently checkable.
- **A source is a normal HTTPS file server.** It hands out chunks by name, nothing more. One command turns any machine into one: no account, no daemon, no torrent client.
- **The signed manifest is the map.** It lists the chunks that make up every file, so the client knows exactly what to gather and what each piece must hash to.
- **The client pulls from many sources at once.** It fetches chunks in parallel, checks each one against its hash as it lands, drops and refetches anything that does not match, and resumes where it stopped. Bytes from a lying source are thrown away before they ever touch the model.
- **Shared chunks download once.** Because the cut points follow the content, two versions that share weights share chunks, so moving from one version to the next pulls only what changed.
- **Only model files get in.** A fixed allowlist of weight formats (`.safetensors`, `.gguf`, `.bin`, `.pt`, `.pth`, `.onnx`, `.ggml`, and quantized variants) passes. Anything else is refused when the manifest is built, so a seeder cannot slip a `movie.mp4` into the network.

The result: a model's survival depends on no provider, no single server, and not on us. As long as one web server anywhere holds its chunks, the model is alive, and every byte still proves itself against the signed hashes.

9. Keeping the backend alive

The backend itself never accepts an inbound connection. It dials out to its edge nodes over an encrypted tunnel and answers on that already-open path, so its address is never written to an edge's disk and no port is open to the world. The public only ever touches the edge nodes, which hold no secrets and no copy of the backend's address. Seize an edge and you get a throwaway box and a dead end.

One path is different on purpose: a Tor v3 onion service. Reached through it, the backend has no clearnet address anywhere in kernel or memory, so even a machine seized while still running gives up nothing about where the backend sits. The onion address is itself a public key (a 56-character Ed25519 key, self-authenticating, no certificate authority and no registry to seize), and the service runs the standard hardening against the guard-discovery attacks that threaten a long-lived onion. The everyday tunnel is faster but would leak the current edge addresses under a live seizure; the onion path survives even that.

The backend's address must never leak anywhere public. We treat that as a fixed list of about twenty ways an address escapes, and we close every one: DNS history, TLS certificate logs, default replies on a bare connection, error-page bodies, mail headers, server-side fetches of user-supplied URLs, favicon fingerprints, redirects, subdomain scans, reverse DNS, exposed status and metrics pages, gateway-timeout pages that print the upstream address, and the rest. Error pages are the worst offender, so every response outside the normal range collapses to one identical generic page, rendered at the edge, across the whole 4xx and 5xx range. A build check scans every shipped file and every response for anything address-shaped and fails the build on a single hit.

Names are spread across countries: four domains, four registries, four legal systems. One is the public front door and is assumed seizable. The others are anchors in friendlier jurisdictions, locked at the registry where the registry allows it. No single court reaches all four. DNS is served from a hidden primary that is not named in the zone, out to throwaway secondaries in several countries; the records carry a long expiry, so losing the primary still leaves the name resolving for weeks. The chain is signed (DNSSEC), on a different algorithm and a different key from the one that signs the model hashes, so the two trust roots never mix.

10. Finding a way in

Finding a live entrance is its own blocking problem, so it gets its own signed object: a small signed routing list. It carries the entrances that are alive now, each tagged by type, plus two guards:

- A counter that only ever goes up. The client accepts a new list only if the counter is higher than the highest it has ever seen, and it keeps that number across restarts. This blocks an attacker from replaying an old list.
- A short expiry, one to three days out. A signed-but-stale list is treated as emergency-only.

The website and the client read the exact same list, and both check it the same way: verify the signature against the built-in key, reject it if expired, accept it only if the counter went up, then keep it as the last known good one. Because the list is signed, the channel that carries it does not have to be trusted. A hostile relay can only hand over a valid signed list, which is harmless, or a rejected one.

The list is published over several independent channels (a copy built into the client, signed DNS records, Nostr, content-addressed storage, and plain third-party drops), so losing any one is invisible. The client takes the highest counter across every channel that answers.

It then walks backup routes until one works, in order: the front-door domain, then the sibling domains in other countries, then an optional flood-absorbing skin, then the Tor onion, then content-addressed retrieval, and last a fully offline path, a copy you already hold plus the signed hashes, which works with no live discovery at all. The client moves on at any failure: a refused connection, a DNS error, a timeout, a blank or opaque response. Inside a single route it races the candidates in parallel and keeps the first that answers.

11. Trusting a mirror

A user sees three states: checked, not yet checked, and failed. Only the signed hashes decide which one a file gets. No operator has a button that marks bytes "checked" by hand.

How a mirror earns trust depends on how it serves bytes:

- A content-addressed source (a CID or a v2 infohash) is checked by comparing its address to the signed manifest. Free, instant, and impossible to fake, because the address is the hash of the bytes.
- A plain HTTPS mirror can swap bytes after it is checked, so it is re-checked on a schedule. A checker fetches random byte ranges over HTTP, three to eight slices of 16 to 64 MB per shard, and tests them against a hash tree stored when the file was taken in. The offsets are random every cycle, so "honest at the edges, poisoned in the middle" does not work, and a full re-hash runs now and then so nothing is sampled forever in the same place.

A file whose bytes ever stop matching its hashes is marked failed, for good, and logged in the open. A mirror that is merely offline can come back; tampering cannot be taken back. Gone is recoverable. Lying is not. The checkers run away from the backend, on rotating addresses, and refuse to follow links into private address ranges, so a hostile submitted URL cannot map them or turn them into a probe. The last backstop is the user's own client, which re-hashes locally and cannot be fooled by any mirror.

12. No master, and the client

Anyone can run a full mirror node from public files alone, with no secret from us. A node announces itself by publishing a signed Nostr event, which buys it no trust at all and is only a way to be found. It pulls the catalog by asking for everything changed since a timestamp, a request the project and every other node answer, so there is no single master to seize.

For every version, a node does what the client does: fetch the signed manifest, check the signature against the built-in key, fetch the bytes from any source (the source counts for nothing), re-hash every shard, then store and re-announce. The worst a hostile upstream can do is stall or hold bytes back. It cannot forge, because the signature stops it, and it cannot poison, because the hash stops it. Holding back is beaten by syncing from several upstreams and taking the union. If the founding servers vanish, every running node keeps serving what it holds, the swarms and content-addressed copies survive as long as one machine seeds or pins them, and the catalog rebuilds from the public record.

The client is a single small program with the project's Ed25519 public key built in. Everything it trusts, it checks offline against that key.

```
omm download <model>[@<version>]
omm seed <path>
omm verify <path>
```

`download` finds a live entrance from the signed routing list, fetches the manifest and checks its signature, races the download set for a live source, streams the shards while computing SHA-256 and BLAKE3 in one pass, recomputes the whole-model hash, and on any mismatch drops the bytes and refetches from another route. `seed` turns the bytes you already hold into a source others can pull from, with one command. `verify` re-hashes local files against the signed manifest with no network at all, the floor below which nothing can lie to you. It runs over Tor, runs against a local content address, and runs from a USB stick with no cleartnet. Downloads need no account. Releases are reproducible and ship a detached Ed25519 signature, and updates are opt-in and refuse to go backward.

13. The build order, and the open questions

Trust first, reach second, the hard defenses last, each one added just before the threat that makes it worth the cost.

- **First, the directory.** Mirror every platform's catalog and index every existing download, pointing out with full provenance and the source's own hashes. Useful at once, hosts nothing.

- **Second, the signed archive.** Take in, hash, and sign a chosen core of the most-at-risk models; host it on our chunk transfer, BitTorrent v2, content-addressed storage, and cold storage; ship the one-command source; stand up the first out-of-country anchor domain and the checked badge.
- **Later, the full defense.** The complete multi-country edge spread, the signed routing list with all its backup routes, the backend reached over Tor, and open federation, so no single operator can end the project.

The design has been through an adversarial review, and the security-critical pieces held: the three-layer split, the off-the-internet backend, content addressing, and the multi-country spread. Some choices are still open, and the design says so rather than overclaiming: the legal home of the project, the exact way the signing key is rotated or revoked, the re-check intervals, and which registrars will lock which domains. Survival against a backend seized while still running depends on the Tor path, which lands in a later phase; until then, that one guarantee is not yet in force.

14. Why we build for the dark

Most of this you will never see working, because most of these attacks never come. We build it anyway, and we build the trust core first.

We are paranoid on purpose. We plan for seized servers and poisoned files. We plan for a state that has decided this archive should not exist. And we hope every layer of it sits unused for years. The job is narrow and total: open-source and open-weight models stay downloadable, stay provably real, and stay yours. For anyone. Anywhere. For good.